GENERATIVE AI FUNDAMENTALS · FINAL REPORT

Building a Retrieval-Augmented Generation System

Academic Literature RAG over five foundational Transformer-era papers

An evidence-grounded conversational RAG application built on structure-aware Docling ingestion, hybrid retrieval with cross-encoder reranking, four-turn conversational memory, configurable open-source generation, and a metrics-driven evaluation program that guided every architectural decision described in this report.

5	50	9	+23%
SOURCE PAPERS	EVAL QUESTIONS	CHUNKING EXPERIMENTS	MRR FROM RETRIEVAL UPGRADE

SYSTEM AT A GLANCE	
Source corpus	Attention Is All You Need, BERT, GPT-3, RoBERTa, T5 (five arXiv papers)
Ingestion	Docling layout-aware parsing, table and formula extraction, HybridChunker
Vector database	Qdrant persistent local collection, cosine similarity, metadata payloads
Embeddings	all-MiniLM-L6-v2 (default) and BAAI/bge-base-en-v1.5 (evaluated alternative)
Retrieval	Dense and text-match candidates, cross-encoder rerank, neighbour context expansion
Generation	Configurable through LiteLLM: Ollama, Groq, Google, or OpenAI
Memory	Last four completed user and assistant turns
Evaluation	Self-devised retrieval metrics (MRR, nDCG, coverage) with LLM-as-judge answer scoring

1 Objective

The brief asked for a RAG application that ingests five academic PDFs, indexes them in a vector database for semantic retrieval, generates grounded answers with an open-source language model, maintains memory across the last four turns, and is evaluated on a predefined question set using an established or self-devised framework, with the approach, implementation, and results documented here.

The implementation targets exactly this brief using the five specified papers (Vaswani et al., 2017, *Attention Is All You Need*; Devlin et al., 2018, *BERT*; Brown et al., 2020, *GPT-3*; Liu et al., 2019, *RoBERTa*; Raffel et al., 2019, *T5*). It also exceeds the minimum bar in several places: the evaluation suite holds 50 questions rather than 10, retrieval quality was tuned through nine chunking experiments before the final configuration was fixed, and deterministic input and output guardrails were added once the core pipeline was reliable.

Design principles that guided every decision

- **Evidence over intuition.** Every non-trivial choice (chunk size, chunking unit, cleaning, embedding model, retrieval strategy) was measured on the same metrics before adoption rather than picked by convention. Section 5 walks through this trail.
- **Traceability.** Every stored chunk carries its source paper, section heading, and page number, so answers can be grounded and audited rather than merely generated.
- **Model portability.** Generation is provider-agnostic through LiteLLM, so the same retrieval pipeline runs against a fully open-source local model (Ollama) or a hosted model without touching retrieval code. This satisfies the open-source-model requirement while keeping hosted options available for comparison.

> **CENTRAL FINDING**
>
> Moving from plain dense retrieval to hybrid candidates with cross-encoder reranking and neighbour context expansion lifted retrieval MRR from 0.63 to 0.77 (about +23%). That gain was larger than any chunk-size or embedding-model change, and it is the central result of the evaluation program (Section 5.4).

2 Overview and System Architecture

The system is split into two pipelines that share only the Qdrant vector store: an offline **ingestion pipeline** that runs when the corpus changes, and an online **query pipeline** that runs per chat turn. This keeps the slow, CPU-heavy ingestion decoupled from the interactive chat path.

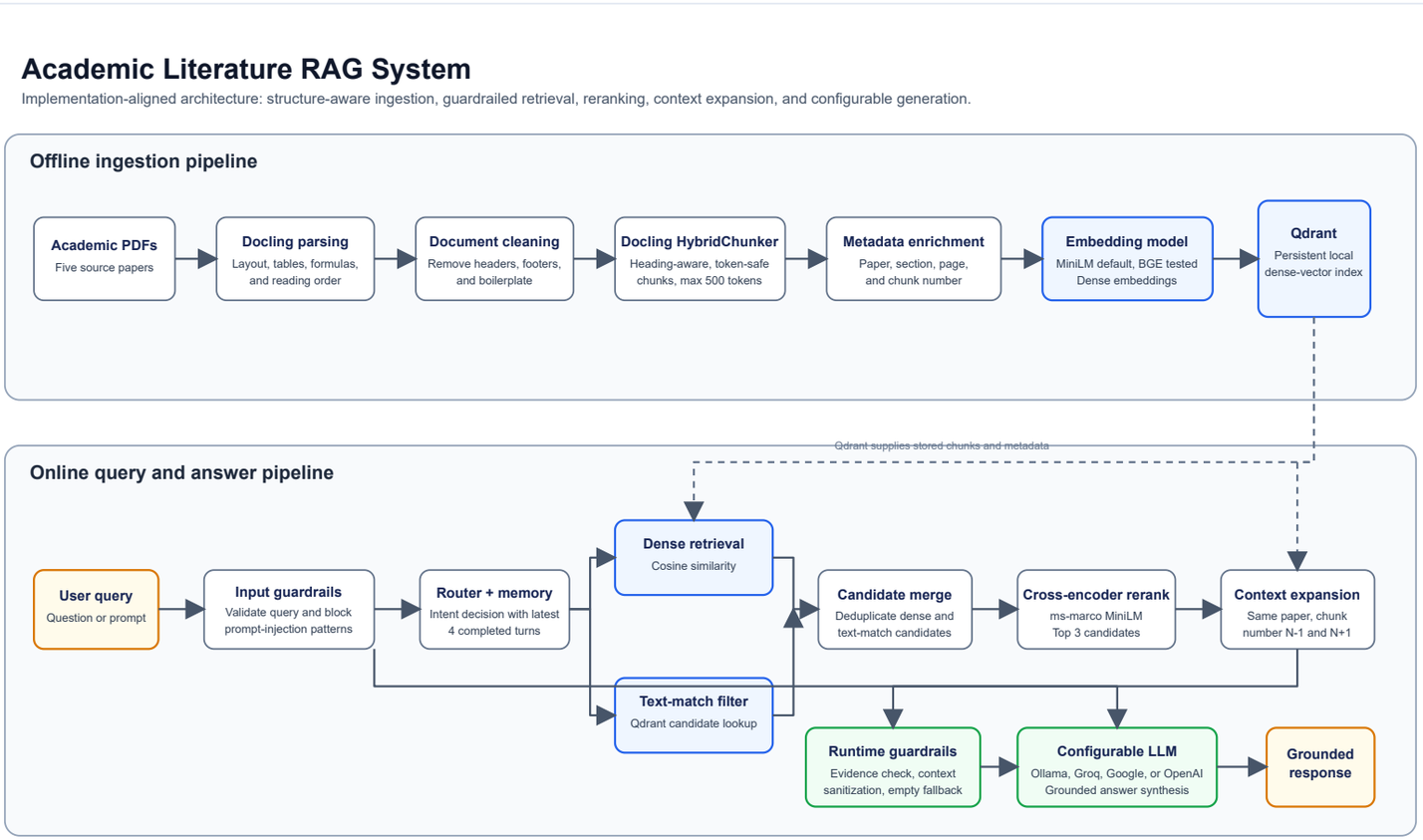


Figure 1. End-to-end architecture. Offline structure-aware ingestion (top) and the guardrailed, memory-aware online query and answer pipeline (bottom).

Offline ingestion, in order

- **Docling parsing** produces a structured document tree with layout analysis, table structure, and formula enrichment.
- **Cleaning** strips headers, footers, arXiv banners, emails, and copyright text.
- **HybridChunker** creates heading-aware, tokenizer-accurate chunks capped at 500 tokens, each prefixed with its heading path.
- **Metadata enrichment** tags paper, section, page, and chunk number.
- **Embedding** produces normalized SentenceTransformer dense vectors.
- **Qdrant upsert** uses deterministic IDs to make re-ingestion idempotent.

Online query, in order

- **Input guardrails** reject malformed queries and block prompt-injection.
- **Router and memory** decide whether retrieval is needed, using the last four turns.
- **Hybrid retrieval** combines dense and text-match candidates, deduplicated.
- **Cross-encoder rerank** scores each query and chunk pair jointly, keeping the top three.
- **Context expansion** stitches each hit together with its neighbour chunks.
- **Runtime guardrails** run an evidence check, context sanitization, and empty fallback.
- **Generation** sends a grounded prompt to the configured LLM provider.

3 Technical Implementation and Rationale

Every major decision below is stated with its reasoning, the alternatives considered, and, where the choice was verified experimentally, a pointer to the result in Section 5.

3.1 PDF extraction: why Docling

The five papers are dense PDFs with multi-column layouts, tables, mathematics, and figures. A naive extractor such as PyPDF2 reads them as an unordered text stream that interleaves columns, drops table structure, and loses section hierarchy, all of which matter for chunking by meaning.

> **◆ DECISION: PDF PARSER**
>
> Docling was chosen over PyPDF2 and pdfminer.six because it performs layout analysis (reading order and columns), structured table extraction (`do_table_structure=True`), and LaTeX formula enrichment (`do_formula_enrichment=True`), exposing a structured document tree rather than flat text. That structure is what later enables heading-aware chunking and per-chunk section metadata. See `src/ingestion/docling_pdf_loader.py` .

Because Docling parsing is CPU-intensive, each parsed document is cached to `docs/parsed_json` . A fast `--loader json` mode reuses the cache, while `--loader pdf` re-parses from source.

3.2 Preprocessing and cleaning

Docling's own header, footer, and page-number labels blank the running furniture. A regex pass then removes conference and preprint boilerplate, emails, and copyright notices, and a BeautifulSoup pass strips stray HTML artifacts (`src/ingestion/docling_doc_cleaner.py`).

> **◆ DECISION: CLEAN BEFORE CHUNKING**
>
> Cleaning runs before chunking, so boilerplate never occupies a slot inside the 500-token budget or gets embedded as content. The effect is measurable: at a matched 1000-character chunk size, cleaned markdown scored MRR 0.519 and nDCG 0.576 against 0.502 and 0.564 for raw markdown, and yielded one more usable test case (Section 5.2).

3.3 Chunking: why HybridChunker, and why 500 tokens

Chunking has the largest measured effect on retrieval quality in this project (Section 5.1). Naive character splitting was compared directly against the Docling `HybridChunker` , which chunks along structural boundaries and enforces an exact tokenizer budget matched to the embedding model, merging small adjacent units.

> **◆ DECISION: CHUNKING UNIT AND SIZE**
>
> Token-based HybridChunker output was adopted over character splitting, at 500 tokens. Across a 500 to 3300 character sweep, keyword coverage never exceeded 79 percent. Switching to token-aware chunks lifted coverage to between 86.5 and 91.7 percent at every size, while matching or beating the best character-based MRR and nDCG. Within the token sweep, 500 tokens gave the best balance (MRR 0.625, nDCG 0.645, coverage 91.7 percent) and was fixed as `TARGET_MAX_TOKENS` .

3.4 Embedding model: MiniLM default, BGE-base evaluated

SentenceTransformer models were used rather than a hosted embedding API, keeping indexing fully open-source. Two models were compared on the identical optimised pipeline: `all-MiniLM-L6-v2` (384-dimensional, fast) and `BAAI/bge-base-en-v1.5` (768-dimensional).

> ◆ **DECISION: EMBEDDING MODEL**
>
> MiniLM stays the default. On the final pipeline the two were near-identical: MRR 0.771 against 0.764, nDCG 0.773 against 0.776, and answer accuracy 5.00 against 4.94 out of 5. The 2x larger vector size and slower CPU inference of BGE-base were therefore not justified for this corpus. BGE-base remains a one-line configuration change.

Embeddings are L2-normalized (`normalize_embeddings=True`) so Qdrant's cosine distance is exact.

3.5 Vector database: why Qdrant over FAISS

The brief names FAISS as suggested, not mandatory. FAISS is an in-process search library with no native persistence, metadata filtering, or server mode, so using it would mean hand-rolling a metadata store and disk persistence alongside it.

> ◆ **DECISION: VECTOR STORE**
>
> Qdrant provides all three needed capabilities natively: persistent on-disk storage (`QdrantClient(path=...)`, with no server needed locally), rich per-point JSON payloads (paper, section, page, chunk number) queryable through filters, and a text-match scroll query that supplies the lexical half of hybrid retrieval, all from one client. The same code path supports a remote cluster through `QDRANT_URL`. See `src/ingestion/vector_store.py`.

Deterministic IDs based on `uuid5(paper_name, chunk_number)` make re-ingestion idempotent, updating chunks instead of duplicating them.

3.6 Retrieval: hybrid candidates, reranking, context expansion

Retrieval evolved in three additive stages, each with a measurable gain (Section 5.4):

- **Hybrid candidates.** Dense cosine search misses exact-term matches such as an acronym like "NSP". Qdrant's text-match query supplies lexical candidates in parallel, deduplicated by ID.
- **Cross-encoder reranking.** A cross-encoder (`ms-marco-MiniLM-L-6-v2`) reads each query and chunk pair jointly, a stronger signal than bi-encoder cosine. The top 20 candidates rerank down to the top 3.
- **Neighbour context expansion.** Each reranked chunk is stitched together with its preceding and following chunk in the same paper, so the model sees continuous prose rather than an isolated slice.

> **DOCUMENTED SCOPE LIMIT**
>
> This uses a Qdrant text-match candidate path rather than a full BM25 inverted index with reciprocal-rank fusion. The measured gain was already large (Section 5.4), so full BM25 with RRF is a documented future improvement rather than a blocking requirement.

3.7 LLM integration: open-source through LiteLLM

Generation and routing go through LiteLLM (`src/helpers/llm_helper.py`), which normalizes calls across providers behind one interface.

◆ DECISION: LLM PROVIDER

Ollama (fully local, open-weight `llama3.2`) runs with zero external dependency and satisfies the requirement directly. The checked-in default is Groq (`groq/openai/gpt-oss-120b`), an open-weight model served over a fast hosted API that keeps the demo responsive without a local GPU while remaining open-weight rather than closed. Switching `LLM_PROVIDER` needs no code change.

The router's intent classifier is pinned to a separate, fast model (`gemini-3.1-flash-lite`), so switching the answer model does not degrade routing. Hosted calls get bounded retries, while local Ollama gets a single attempt to fail fast within its timeout.

3.8 Conversational memory: four-turn window

`retain_recent_completed_turns` returns `chat_history[-8:]`, the last four turns as eight messages. The trim is applied once at the top of `answer_query`, so routing and generation share one consistent memory. The Gradio panel still shows the full transcript; only the model-facing context is bounded.

3.9 Guardrails and response refinement

Guardrails were added after the core pipeline was validated, once testing surfaced empty or oversized queries, prompt-injection (in the query and inside retrieved text), thin-evidence retrieval, and empty LLM responses. All checks are deterministic (regex and length), not a second LLM call, keeping them fast and fully inspectable (`src/helpers/guardrails.py`). A companion router path detects follow-ups such as "make that shorter" or "as bullet points" and rewrites the previous answer without re-retrieving.

4 Evaluation Methodology

A 50-question suite (`src/evaluation/tests.jsonl`) was built instead of the minimum 10, spanning all five papers and six categories, each annotated with a reference answer and a keyword set for automatic grading.

Dimension	Breakdown
By paper	GPT-3: 16 · Attention: 10 · RoBERTa: 10 · BERT: 7 · T5: 7
By category	motivation: 16 · mechanism: 12 · single_doc_multihop: 9 · direct_fact: 6 · definition: 6 · reasoning: 1
By difficulty	medium: 21 · hard: 17 · easy: 12

4.1 Representative test questions

Paper	Category and difficulty	Question
Attention	motivation · easy	What is the primary motivation for introducing the Transformer architecture?
BERT	direct_fact · easy	What are the two pretraining objectives introduced in the BERT paper?
RoBERTa	definition · medium	What is dynamic masking, and how does it differ from BERT's masking strategy?
RoBERTa	mechanism · medium	Why does RoBERTa increase the amount of pretraining data compared with BERT?
T5	motivation · medium	Why does T5 systematically compare transfer-learning approaches instead of proposing one architecture?
GPT-3	definition · easy	What does the GPT-3 paper mean by "in-context learning"?
GPT-3	direct_fact · medium	What concern does GPT-3 raise regarding data contamination in benchmarks?
GPT-3	multihop · hard	Why do the authors argue in-context learning is not equivalent to learning a task from scratch at inference?

4.2 Retrieval metrics (self-devised)

With reference answers available but no gold "relevant chunk" labels, retrieval is graded with a keyword-grounded proxy (`src/evaluation/eval.py`). The brief explicitly permits a documented self-devised metric:

- **MRR.** For each keyword, 1 divided by the rank of the first chunk containing it (0 if absent), averaged across a question's keywords.
- **nDCG.** Binary keyword relevance per rank, log-discounted and normalized, which rewards evidence near the top of the list.
- **Keyword coverage.** The percentage of a question's keywords found anywhere in the top-k.

These are lexical proxies, chosen because they are cheap, deterministic, and reproducible enough to run as an inner loop across nine configurations, which an LLM-judge retrieval metric would make prohibitively slow.

4.3 Answer evaluation: LLM-as-a-judge

Generated answers are graded against the reference by an independent judge (`gemini-3.1-flash-lite`, deliberately different from the generator to avoid self-preference), with Pydantic-enforced structured output on three dimensions scored 1 to 5: **accuracy** (any wrong statement forces a 1), **completeness** (a 5 requires full coverage), and **relevance** (a 5 requires no extraneous content). These map onto the brief's four aspects: relevance and response quality through the judge scores plus manual inspection, accuracy against the source PDFs, and contextual awareness through the four-turn memory and refinement flow.

5 Results and Discussion

Every change below was measured on the same 50-question suite and the same metrics, so the numbers are directly comparable. This is the trail that produced the final configuration in Section 3.

5.1 Experiment 1: chunking unit and size

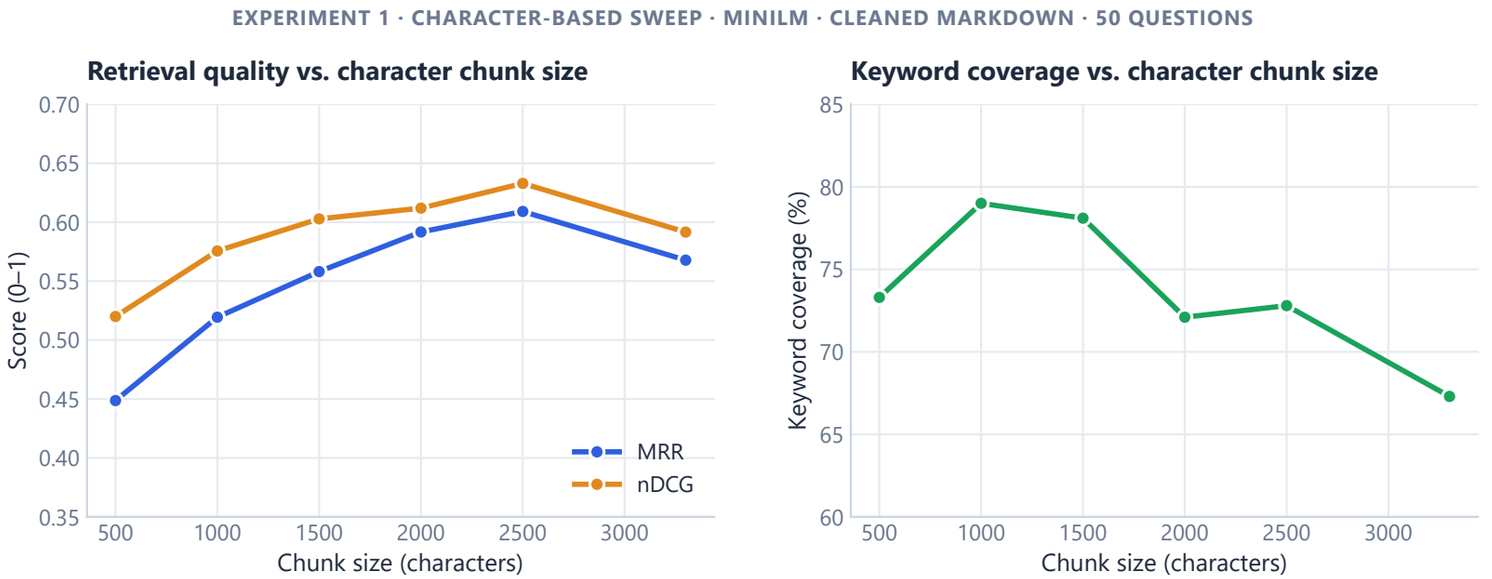


Figure 2. Character chunking. MRR and nDCG peak at 2500 characters then degrade at 3300, and coverage never exceeds 79 percent.

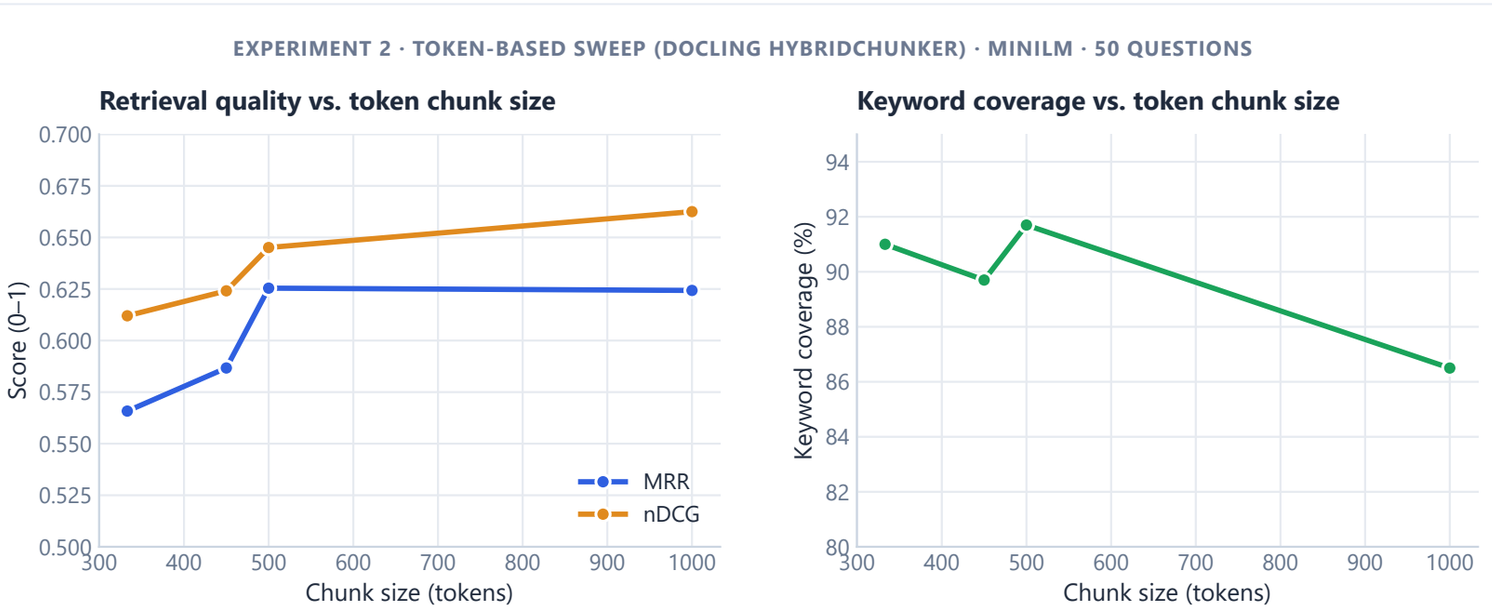


Figure 3. Token chunking clears 86 percent coverage at every size, 7 to 25 points above any character configuration, with 500 tokens giving the best balance of MRR, nDCG, and coverage.

Character splitting ignores sentence and section boundaries, truncating mid-sentence, which explains both the coverage ceiling and the drop at 3300 characters. Token chunking chunks along structure first, then enforces the budget, and prepends each chunk's heading path. A size of **500 tokens** was adopted.

5.2 Experiment 2: does cleaning help?

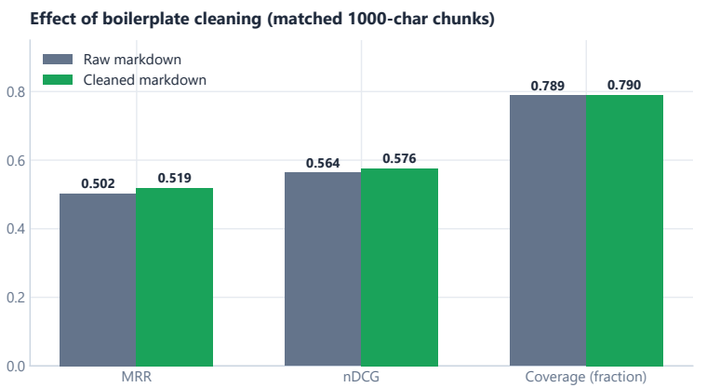


Figure 4. At matched 1000-character chunks, cleaning improves MRR and nDCG and yields one more usable test case.

The gain is modest in absolute terms (MRR up 0.017, nDCG up 0.012) but consistent across every metric, and the raw-markdown run produced one fewer usable test case. This is evidence that boilerplate occasionally consumed enough of a small chunk's budget to displace the substantive content. It justified keeping the cleaning step even though its effect is smaller than the chunking-strategy change in Experiment 1.

5.3 Experiment 4: retrieval-pipeline upgrade

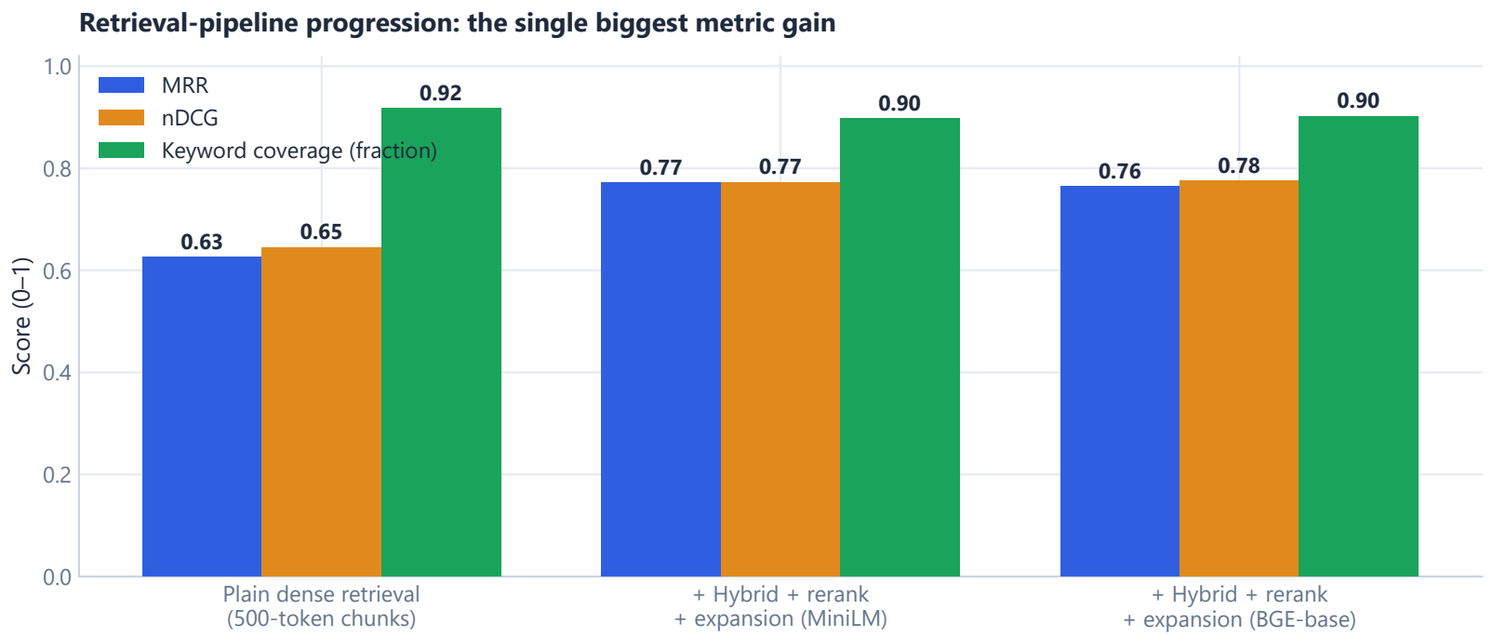


Figure 6. Hybrid candidates, cross-encoder reranking, and neighbour context expansion, applied on top of the best plain configuration, gave the single largest metric gain in the project.

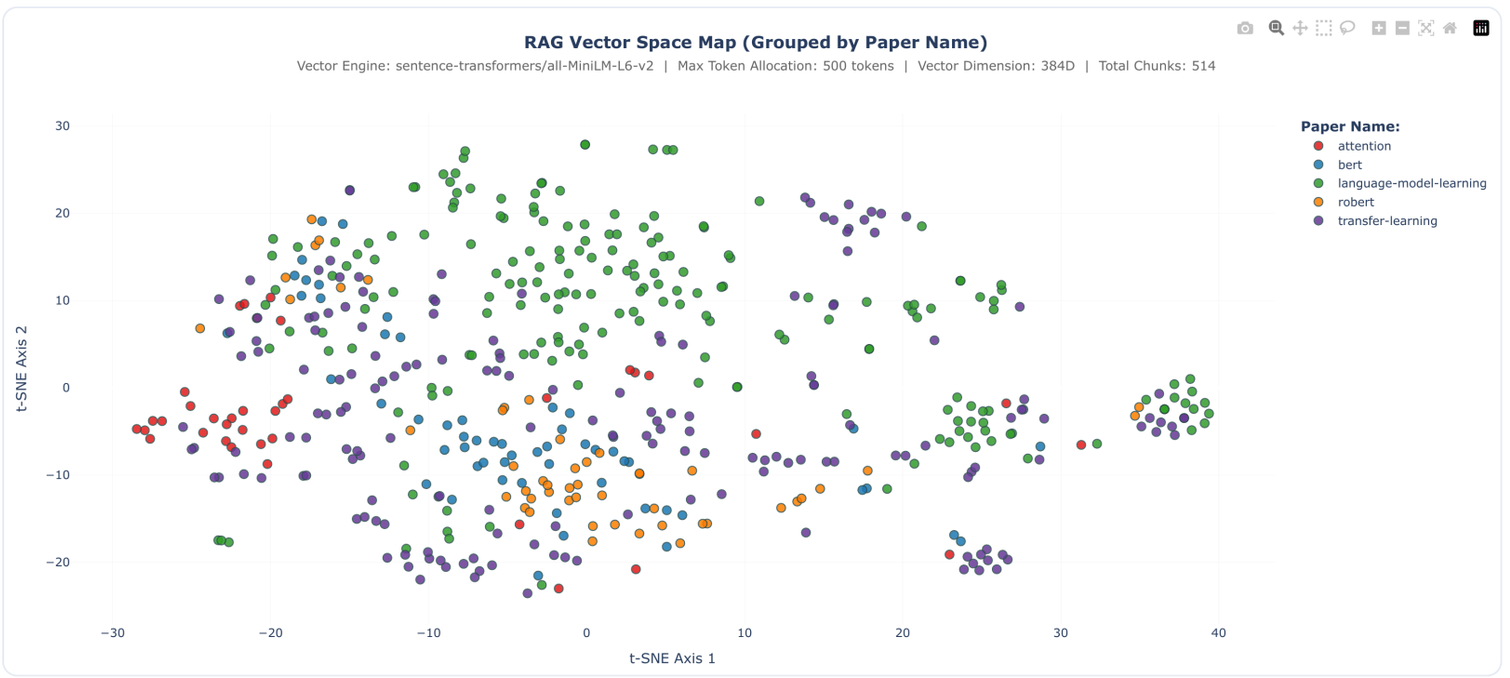
MOST CONSEQUENTIAL RESULT

MRR rose from 0.625 to above 0.77 (about +23 percent) and nDCG from 0.645 to above 0.77 (about +20 percent) by upgrading retrieval alone, with no change to chunk size, embedding model, or LLM. Single-signal dense retrieval is the weakest link once chunking is tuned, and a cross-encoder reading the actual query and chunk pair captures relevance that vector similarity cannot. Coverage stayed flat (around 89 to 92 percent) because it measures whether keywords appear anywhere in the top-k, which reranking does not change, whereas MRR and nDCG measure where the best evidence lands, which is exactly what reranking optimises.

5.4 Experiment 3: embedding-space separation

Beyond keyword metrics, the embedding space was inspected directly. Chunk vectors were reduced to 2D with t-SNE and colored by source paper (`src/vector_space_visualizer.py`). Well-separated clusters indicate the model captures paper-level topical distinctions.

MiniLM-L6-v2 · 384 dimensions



BGE-base-en-v1.5 · 768 dimensions

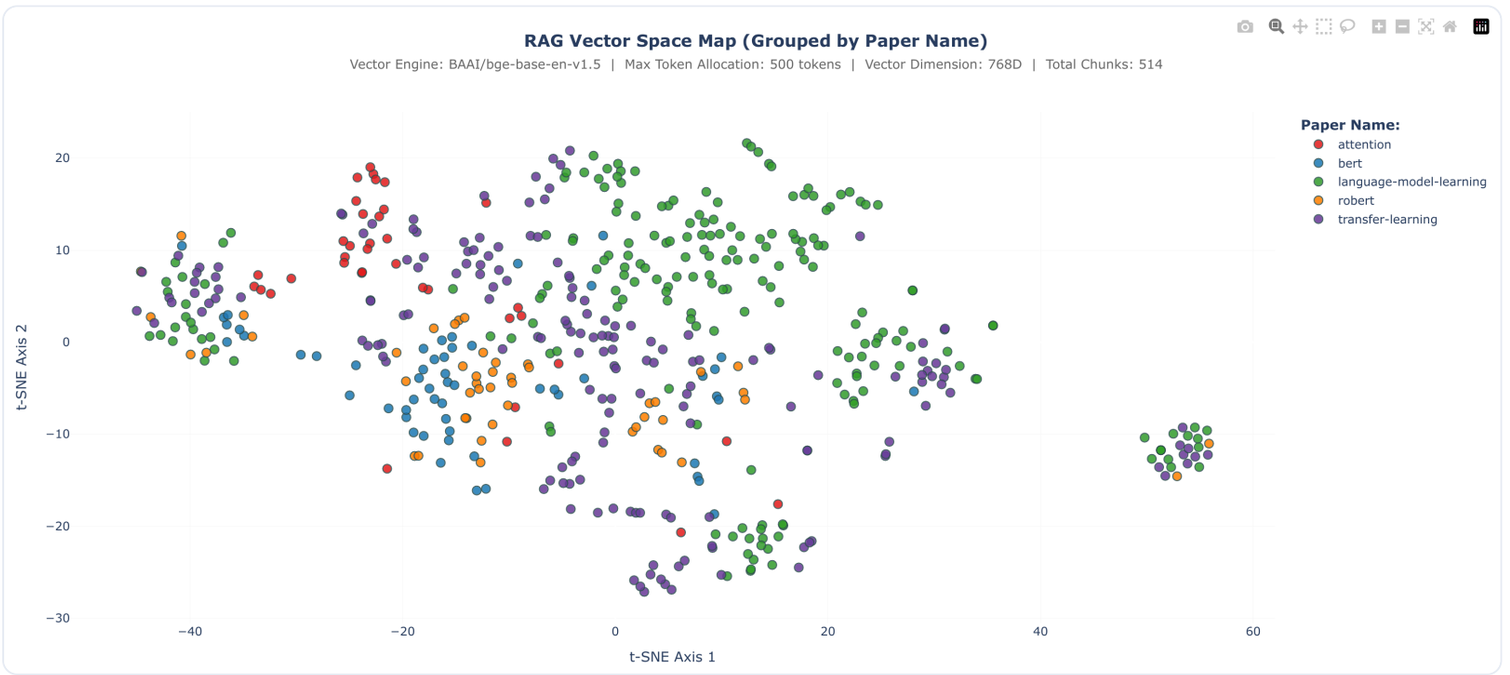


Figure 5. The same 500-token chunk set under MiniLM at 384 dimensions (top) and BGE-base at 768 dimensions (bottom). BGE-base produces a visibly tighter cluster structure, yet Section 5.5 shows this did not translate into a materially better end-to-end score once reranking was in the pipeline.

5.5 Experiment 5: embedding model and answer quality

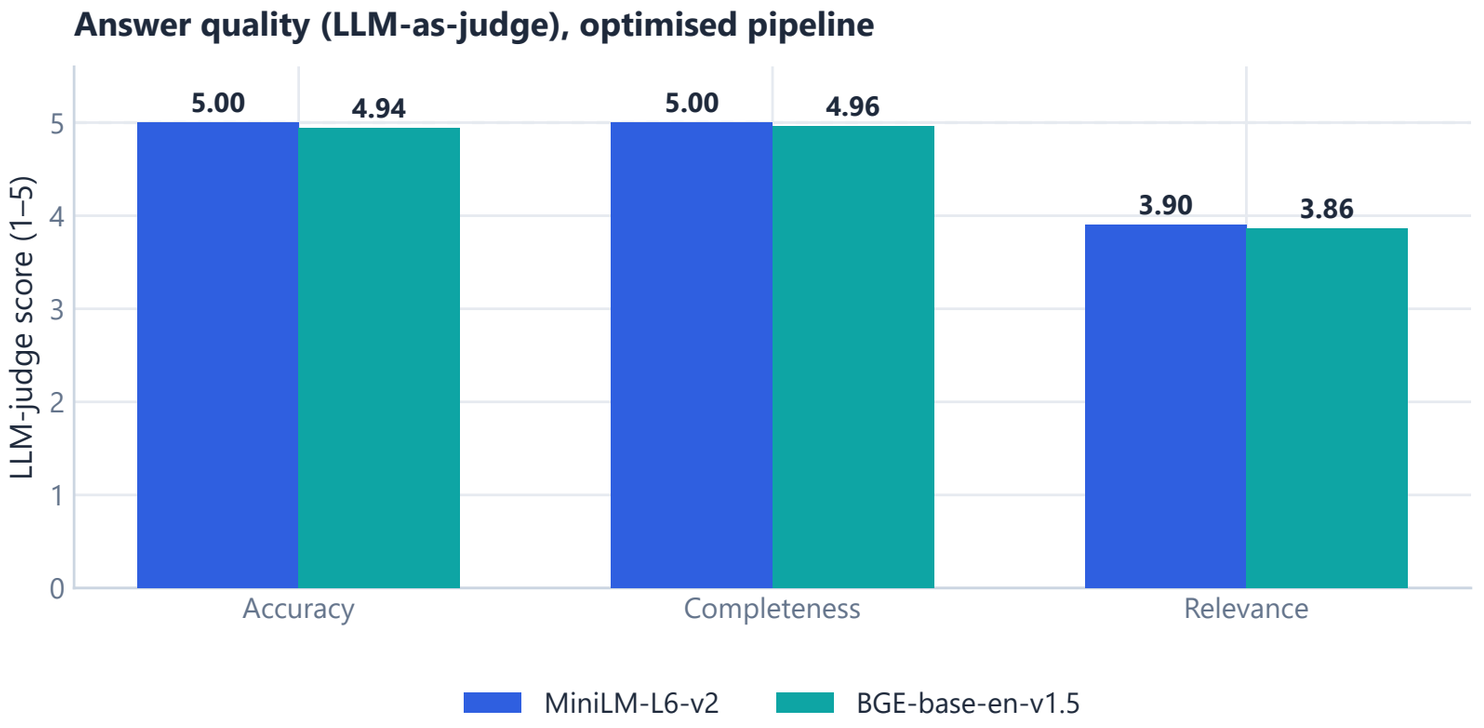


Figure 7. LLM-judge answer quality on the optimised pipeline. Accuracy and completeness are near-maximum for both models, while relevance is the clear outlier, roughly a full point lower.

With retrieval already strong, the two embedding models gave near-identical answer quality (accuracy 5.00 against 4.94, completeness 5.00 against 4.96). The one dimension that did not saturate was **relevance** (3.90 and 3.86): the judge penalizes content beyond what was asked, and the grounding prompt currently favours thoroughness over strict scope. That is a concrete, metric-identified target for prompt tuning (Section 5.7) rather than a vague impression.

5.6 Challenges faced

- **Windows symlink restriction.** Hugging Face caching triggers `WinError 1314` without elevated privileges, resolved by setting `HF_HUB_DISABLE_SYMLINKS=1` before importing Docling.
- **Keyword-metric sensitivity.** Exact-keyword matching scored some genuinely relevant retrievals as 0 when the reference used a synonym, a known limitation carried openly into Section 4.2.
- **LLM-eval variance.** The same optimised MiniLM configuration produced MRR of 0.7504 and 0.7710 across two runs, so this non-determinism is treated as noise at the 0.01 to 0.02 scale.
- **Relevance against thoroughness.** The prompt's instruction to synthesize into one coherent answer aids completeness but measurably lowers the judge's relevance score.
- **Cost against iteration speed.** A full answer evaluation invokes the router, generator, and judge for all 50 questions, so the cheaper keyword metrics were the inner-loop signal across all nine chunking experiments, with the LLM judge reserved for the final configuration.

5.7 Potential improvements

- A true BM25 sparse index with reciprocal-rank fusion, replacing the Qdrant text-match path.

- Tighten the grounding prompt's scope discipline to close the relevance gap in Figure 7, then re-run answer evaluation.
- Send extracted figure images (saved in `docs/extracted_images`) to a vision LLM for figure summaries indexed alongside text.
- Replace keyword-substring grading with embedding-similarity relevance for synonym cases.
- Implement the bonus feedback loop (Section 7).

6 Repository, Setup, and Applications

All source code for this application lives in a public GitHub repository. Its `README.md` is the authoritative setup guide and covers environment creation, provider credentials, corpus ingestion, and how to run each application.

`https://github.com/skdhanjal/rag-system`

6.1 Setup, in brief

The project uses `uv` for dependency management. The README documents the full flow; the essential steps are:

1. Install dependencies with `uv sync`.
2. Create a `.env` file and add credentials for the provider or providers in use (for example a Groq or Google key), following the template in the README.
3. Ingest the supplied pre-parsed corpus with `uv run python -m src.ingestion.ingestion_pipeline --loader json`, which cleans, chunks, embeds, and writes vectors into the local Qdrant collection.
4. Launch either application, described below.

Configuration is centralised in `src/config/settings.py`, covering the embedding model and vector dimension, chunk-token budget, retrieval and reranking depth, memory window, LLM provider, and guardrail thresholds. Full ingestion, provider, and troubleshooting notes are in the README.

6.2 Two Gradio applications

The system ships with two separate browser applications built on Gradio, each serving a distinct purpose:

Chatbot interface

The conversational RAG assistant. It answers questions about the five papers, keeps the four-turn memory window, and shows the retrieved source context (paper, section, and excerpts) beside each answer so responses can be traced back to the corpus.

```
uv run python src/chat_ui.py
```

Evaluation dashboard

The evaluation harness. It runs the retrieval metrics (MRR, nDCG, keyword coverage) and the LLM-as-judge answer scoring across the full test suite, with per-category breakdown charts, and it produced the results reported in Section 5.

```
uv run python src/eval_ui.py
```

A command-line evaluator (`uv run python src/evaluation/eval.py <row>`) runs both retrieval and answer evaluation for a single question, which was used for fast iteration while tuning a configuration change.

7 Conclusion

The implementation satisfies every core requirement. The five specified arXiv papers are ingested through a layout-aware, table and formula-preserving pipeline; a persistent Qdrant database stores traceable, metadata-rich chunks; a configurable LLM layer generates grounded answers and can run entirely open-source through Ollama; the bot bounds its working memory to the last four turns while keeping the full transcript visible; and a 50-question suite grades both retrieval and answer quality with a documented, self-devised methodology.

What sets it apart from a minimum-viable pipeline is that almost every configuration choice was reached by measuring competing options and keeping the winner. The largest lever was the retrieval strategy itself: hybrid candidates with reranking and context expansion improved MRR by about 23 percent, which was more than chunk-size and embedding-model tuning combined. The clearest remaining weakness, also metric-identified, is answer relevance: the system is accurate and complete but occasionally more expansive than asked, which is now a concrete target. Guardrails and a refinement flow harden the system against malformed input, prompt injection, and empty-evidence hallucination with no meaningful latency, since all checks are deterministic. The one component not yet built is the bonus feedback loop, documented below with a concrete path.

8 Bonus: Feedback-Driven Memory

> **STATUS: NOT IMPLEMENTED**
>
> No feedback-capture UI or feedback-weighted memory exists yet in `src/chat_ui.py` or `src/helpers/conversation_memory.py`. This is reported honestly.

The groundwork makes this a scoped next step rather than an open-ended one. The guardrails module already validates and sanitizes turn-level input and output, and the memory module already isolates what enters the model's context behind one function. A feedback loop would add a thumbs-up or thumbs-down control, or a short free-text control, per assistant turn in the Gradio UI; store that signal alongside the turn; and have `retain_recent_completed_turns` weight or annotate low-rated turns, for example by appending a system note such as "the user marked the previous answer unhelpful because ...", so the next generation call is nudged away from repeating the mistake, without retraining any model.

9 Assignment Requirement Mapping

Requirement	Sub-item	How it is addressed
1 · PDF ingestion	Data extraction	Docling layout-aware parsing of all five specified arXiv PDFs with table and formula extraction. Section 3.1
	Preprocessing and chunking	Boilerplate cleaning with the Docling HybridChunker, tuned through nine experiments to 500 tokens. Sections 3.2, 3.3, 5.1, 5.2
2 · Vector database	Chunks, embeddings, DB	MiniLM and BGE embeddings in a persistent Qdrant collection with cosine distance and full metadata payloads. Sections 3.4, 3.5
3 · Open-source LLM	Model and usage	LiteLLM-routed generation. Ollama (fully local, open-weight) supported, with Groq gpt-oss-120b as the default. Section 3.7
4 · Memory	Four-turn window	`retain_recent_completed_turns` bounds model context to the last four turns; the full transcript stays visible. Section 3.8
5 · Interaction and eval	Test suite (10 or more)	A 50-question suite across all five papers and six categories. Section 4.1
	Eval framework	Self-devised retrieval metrics with LLM-as-judge answer scoring, methodology documented. Sections 4.2, 4.3
6 · Final report	This document	Overview, implementation rationale, methodology, results with charts, discussion, and conclusion.
7 · Bonus	Feedback loop	**Not implemented**, documented with a concrete path. Section 7

Repository: `src/` (ingestion, retrieval, evaluation, config, prompts, helpers), `docs/` (source PDFs, parsed Docling JSON, evaluation exports, embedding-space visualizations), `README.md` (setup and run), and `pyproject.toml` with `uv.lock` (dependencies). Public repository at `https://github.com/skdhanjal/rag-system`.